

Title	AXI Register Slice
Project	None
Description	AXI Register Slice의 설계및 동작
Author	holelee
Date	17/Mar/2005

(*) Register Slice의 용도

AXI BUS의 설계에 있어서 기본적인 data의 경로는 모두 combinational logic으로 구성되도록 하고 있다. 그에 따라서 AXI master에서 AXI slave로 연결되는 signal이 decoder/arbitrer등을 지나면서 상당히 많은 delay를 가지게 되는데, 이 delay 때문에 BUS 전체의 clock이 떨어지는 것을 막기 위해서, signal 사이에 register를 timing을 isolation해야 한다. 이런 일을 하는 것을 ARM에서 지칭한 대로 register slice라고 부르기로 한다.

AXI의 5개 channel 모두 VALID/READY handshake를 사용하고, 단순히 전달되는 information의 양만 달라지게 되므로 5개의 channel에 대해서 register slice를 따로 설계할 필요는 없고, 설계한 것 하나를 parameter로 주어지는 bit width에 따라서 instantiation을 함으로써 구현할 수 있다.

또한 register slice는 특별히 어떤 위치에 있어야 하는 것은 아니고, AXI Master 연결 부분, AXI Slave 연결 부분, BUS 내부의 Master/Slave 사이의 연결 어디에도 존재할 수 있고, BUS 사이의 연결이 길어지는 경우에는 여러개를 동시에 사용할 수 있다. Register Slice를 사용하게 되면 data handshake가 1 cycle의 latency(through-put은 1 data/1 cycle로 동일하다.)을 가지지만 timing isolation이 가능하므로 BUS 전체의 clock은 향상된다.

(*) ARM에서 제공하는 Register Slice

ARM AXI에서 제공하는 Register Slice는 기본적으로 다음과 같이 세 가지 동작중에 하나를 하게 된다.

- Fully registered

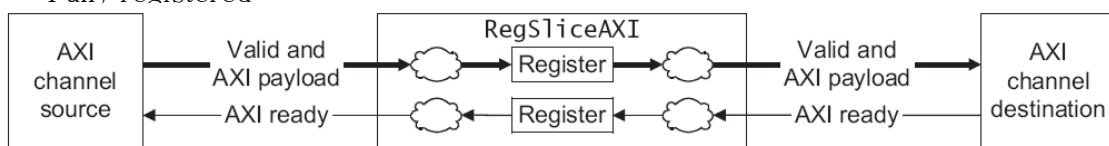


Figure 2 Fully registered configuration

- Registered forward path

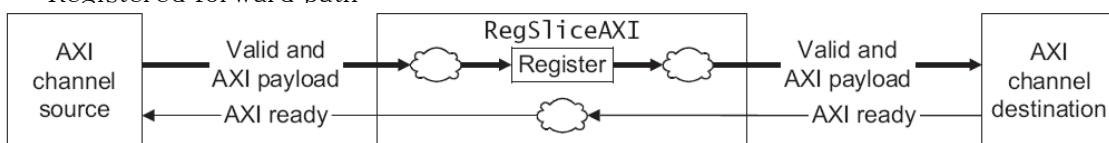


Figure 3 Registered forward path configuration

- Static bypass

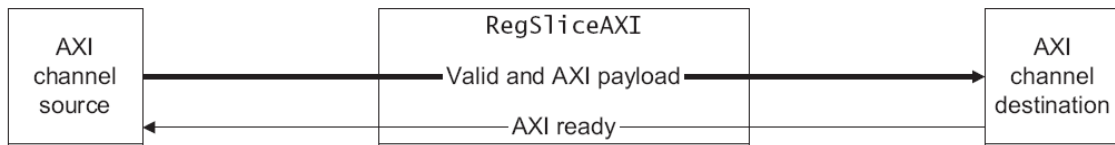


Figure 4 Static bypass configuration

(*) Static Bypass Register Slice

Static Bypass Register Slice는 단순히 Receiver와 Sender의 signal을 연결해 놓은 것일 뿐이고, 실제 register가 들어 있어 timing isolation이 일어나지 않는다. Static bypass register는 굳이 사용할 필요가 없고, 구현의 필요도 없으므로 구현하지 않는다.

(*) Registered Forward Register Slice

Registered Forward Register Slice의 경우 channel을 통해 전달되는 information과 VALID signal은 registered되지만 READY신호는 combinational path를 지니게 된다. READY signal의 timing은 괜찮지만 나머지 information의 timing에 문제가 있는 경우에 사용할 수 있다.

기본적으로 Information을 담은 register와 valid를 저장할 register가 존재하고, register에 valid한 information이 담겨 있지 않는 경우나, Receiver쪽에서 READY가 high인 경우에는 registered되면서 information이 전달되게 된다. 다음과 같이 구현하면 될 것으로 보인다.

```

assign READY_S = (VALID_R == 1'b0 || READY_R == 1'b1) ? 1'b1 : 1'b0;

always @(negedge ARESETn or posedge ACLK)
    if(!ARESETn)
        begin
            VALID_R <= 1'b0;          // VALID LOW when reset : one of AXI
requirements.
            INFORMATION_R <= {Information_width{1'b0}}; // don't care when
VALID_R is LOW.
        end
    else
        begin
            if(VALID_S == 1'b1 && READY_S == 1'b1)
                INFORMATION_R <= INFORMATION_S;
        end
  
```

```

        if (VALID_S == 1'b1)
            VALID_R <= 1'b1;
        else if (READY_R == 1'b1)
            VALID_R <= 1'b0;
        end
    end
end

```

우선 _S는 Sender쪽 signal _R은 Receiver쪽 signal을 의미한다. 정보는 Sender가 VALID_S를 보내고 READY_S를 받을 때 latch하고, READY_S는 두가지 경우에 assert하게 된다. 우선 Receiver가 정보를 받을 수 있을 때(READY_R == 1'b1), 그리고 현재 flipflop이 idle할 때 (VALID_R == 1'b0) 이다. VALID_R는 sender가 VALID_S를 assert했을 때 assert되고, Sender가 더 이상 데이터를 보내지 않고(VALID_S == 1'b0), Receiver가 정보를 accept했을 때(READY_R == 1'b1) 다시 de-assert된다.

(*) Fully Registered Register Slice

Fully Registered Register Slice는 정보 뿐 아니라 READY도 registered되어 전달된다. READY가 registered되므로, Receiver가 보낸 READY 신호는 Sender쪽으로 1 cycle의 latency를 가지고 전달되게 된다. READY가 high로 유지되다가 READY가 low로 가는 경우에 대해서는 Sender는 1 cycle delay를 가지는 READY를 받게 되므로 그 1 cycle의 delay동안 Sender가 VALID한 정보를 보내는 경우 그것을 latch해야 한다. 따라서 information을 담게 되는 register는 depth 2의 que의 형식으로 구현되어야 한다. 다음과 같이 구현될 수 있다.

```

assign INFORMATION_R = INFORMATION_0;
assign READY_S = (ready_r_ff == 1'b1 || que_len == 1'b0) ? 1'b1 : 1'b0;
always @(negedge ARESETn or posedge ACLK)
    if (!ARESETn)
        begin
            VALID_R <= 1'b0;
            INFORMATION_0 <= {WIDTH{1'b0}};
            INFORMATION_1 <= {WIDTH{1'b0}};
            que_len <= 1'b0;
            ready_r_ff <= 1'b1;
        end
    else

```

```

begin
    if(que_len == 1'b0 && !(VALID_R == 1'b1 && READY_R == 1'b
0))

        INFORMATION_0 <= INFORMATION_S;
    else if(que_len == 1'b1 && READY_R == 1'b1)
        INFORMATION_0 <= INFORMATION_1;

    if(VALID_S == 1'b1 && READY_S == 1'b1)
        INFORMATION_1 <= INFORMATION_S;

    if(VALID_S == 1'b1 && VALID_R == 1'b1 && READY_R == 1'b0)
    begin
        que_len <= 1'b1;
    end
    else if(que_len == 1'b1 && READY_R == 1'b1)
    begin
        que_len <= 1'b0;
    end

    ready_r_ff <= READY_R;

    if (VALID_S == 1'b1)
        VALID_R <= 1'b1;
    else if(que_len == 1'b0 && READY_R == 1'b1)
        VALID_R <= 1'b0;

    end
endmodule

```

위의 source는 VALID_R과 INFORMATION_R이 combinational path를 통하지 않고 register에서 직접 출력 된다. 이렇게 하면 Receiver쪽의 timing에 여유를 더 둘 수 있게 된다. Receiver쪽이 아니라 Sender쪽의 timing에 더 여유를 둘 수 있도록 coding을 할 수 있지만, 지금은 구현하지 않도록 한다.

(*) Register Slice 사용법

Register Slice의 source code는 rtl 디렉토리에 registered_forward.v와 fully_registered.v로 존재한다. Modelsim directory에서 simulate.sh를 수행하면 rtl source code와 test bench가 compile되고 simulation된다. simulation은 random한 data를 Sender가 보내주고 receiver가 정확하게 data를 받았는지 체크하도록 되어 있다.

registered_forward.v와 fully_registered.v를 사용하기 위해서는 parameter인 WIDTH값을 각 channel별로 필요한 data의 양에 맞추어준다음, data순서에 맞추어 port mapping을 수행하면 된다.